Assignment 2:
Bezier and B-Spline Surfaces

NAME: SHUHAOYU
STUDENT NUMBER: 2024533012
EMAIL: SHUHY2024533012@SHANGHAITECH.EDU.CN

1 INTRODUCTION

I implemented Bézier and B-spline surface evaluation and rendering using OpenGL and the Arietta math library.

The four model keys select different assets: 1 selects a Bézier teapot with a bottom, 2 selects a Bézier elephant, 3 selects a B-spline teapot without a bottom, and 4 selects a B-spline face. The elephant asset is named `BezierGumbo`, and the face asset is named `BSplineHead`.

The main parts of my implementation are:

- Evaluate Bézier curves with iterative de Casteljau interpolation and extend this calculation to tensor-product patches.
- Evaluate B-spline curves with de Boor's algorithm, including span selection and derivative curves.
- Sample the valid parameter domains, generate triangle meshes, and approximate normals near degenerate boundaries.
- Adapt the tessellation resolution to the relative camera distance.
- Select and drag individual control points through their screen projections.
- Evaluate the supplied bicubic Bézier models in a tessellation evaluation shader.

I use the supplied loading, camera, mesh, and rendering classes as the framework. My main additions are the algorithms in `Bezier.cpp` and `BSpline.cpp`, the adaptive resolution and editing logic in `Main.cpp`, and the evaluation code in `Bezier.tese`.

2 IMPLEMENTATION DETAILS

2.1 Evaluating a Bézier curve

I implemented `EvaluateCurve(controlPoints,t)` in `Bezier.cpp`. Its inputs are the control point array and a parameter $t \in [0, 1]$. It returns one three-dimensional position.

I copy the input into `oldPoints`. Each iteration replaces adjacent points by their linear interpolation and removes the last element. I repeat this process until one point remains. Updating from left to right is suitable because each interpolation reads the following element before it is overwritten. The original control points remain unchanged.

The interpolation formula. For degree p and control points $\mathbf{P}_0, \ldots, \mathbf{P}_p$, I start with $\mathbf{A}_i^{(0)} = \mathbf{P}_i$. At level r,

$$\mathbf{A}_i^{(r)} = (1-t)\mathbf{A}_i^{(r-1)} + t\mathbf{A}_{i+1}^{(r-1)}, \tag{1}$$

where $r = 1, \ldots, p$ and $i = 0, \ldots, p-r$. The result is $\mathbf{C}(t) = \mathbf{A}_0^{(p)}$. At $t = 0$ it is $\mathbf{P}_0$, and at $t = 1$ it is $\mathbf{P}_p$.

The helper `derivation(controlPoints,t)` calculates the tangent at one parameter. For positive degree, I construct derivative control points

$$\mathbf{D}_i = p(\mathbf{P}_{i+1} - \mathbf{P}_i), \qquad i = 0, \ldots, p-1. \tag{2}$$

I evaluate this degree-$(p-1)$ curve using `EvaluateCurve`. The returned value is a vector $\mathbf{C}'(t)$, rather than a new curve object.

2.2 Evaluating a Bézier surface

The function `EvaluateBezier(patch,u,v)` receives one patch and one parameter pair. It returns a `SurfaceSample` containing `position`, `du`, and `dv`. It neither generates the whole mesh nor changes the original control points.

A patch of degrees p and q has $p+1$ points along u and $q+1$ along v. Its expression is

$$\mathbf{S}(u,v) = \sum_{i=0}^{p}\sum_{j=0}^{q} B_i^p(u)B_j^q(v)\mathbf{P}_{ij}, \tag{3}$$

where the B functions are Bernstein basis functions.

For each fixed i, I gather $\mathbf{P}_{i0}, \ldots, \mathbf{P}_{iq}$ and evaluate this curve at v. The resulting points are stored in `Qu`:

$$\mathbf{Q}_i(v) = \sum_{j=0}^{q} B_j^q(v)\mathbf{P}_{ij}. \tag{4}$$

Evaluating `Qu` at u gives the surface position. Applying `derivation` to this same array at u gives $\mathbf{S}_u(u,v)$.

For the other tangent, I gather points with fixed j, evaluate them at u, and store the results in `Qv`. Applying `derivation` to `Qv` at v gives $\mathbf{S}_v(u,v)$. Both orders describe the same surface position, so I calculate `position` only once.

Normals near a pole. At a regular point, I use

$$\mathbf{n} = \frac{\mathbf{S}_u \times \mathbf{S}_v}{\|\mathbf{S}_u \times \mathbf{S}_v\|}. \tag{5}$$

The supplied assets use this order for outward normals. A boundary control row can collapse to one position. At such a pole, the cross product may vanish even though nearby points have useful normals.

I compare the cross-product length with 10^{-12}. If it is too small, `poleVector(patch,u,v)` evaluates nearby parameters and returns the first normalized cross product above the threshold. Corner and boundary searches move into the domain. I use a Bézier step of 10^{-4} and at most 100 attempts in each search direction. This is a nearby normal approximation, rather than an analytic limit. If the CPU search finds no valid candidate, it throws an exception.

© 2026
Computer Graphics I Course Assignment

2.3 B-spline spans and curve evaluation

I implemented `FindSpan` in `BSpline.cpp`. Its parameters are `knots`, `degree`, `count`, and `t`. Here, `count` is the number of control points. Writing it as N and the degree as p, the complete knot vector has $N + p + 1$ entries. The valid domain is

$$[U_p, U_N]. \tag{6}$$

It need not be $[0, 1]$, and the knots need not be clamped.

For an interior parameter, I find k with $U_k \le t < U_{k+1}$. My implementation uses a linear scan. The strict upper comparison skips equal knot values. At the upper domain endpoint, I choose the last nonempty span ending at U_N. Parameters outside the valid domain are rejected.

The function `EvaluateBSplineCurve` receives the control points, knots, degree, and parameter. After finding span k, I copy the $p + 1$ active points, indexed from $k - p$ through k, into the temporary vector `d`. I then apply de Boor interpolation.

The de Boor weights. With global indices, let $\mathbf{D}_i^{(0)} = \mathbf{P}_i$ for the active points. At level r,

$$\begin{aligned} \alpha_i^{(r)} &= \frac{t - U_i}{U_{i+p-r+1} - U_i}, \\ \mathbf{D}_i^{(r)} &= (1 - \alpha_i^{(r)})\mathbf{D}_{i-1}^{(r-1)} + \alpha_i^{(r)}\mathbf{D}_i^{(r-1)}. \end{aligned} \tag{7}$$

The active indices are $i = k - p + r, \ldots, k$. After p levels, the answer is $\mathbf{D}_k^{(p)}$.

My code packs each shrinking level at the beginning of `d`. Therefore, the final value is returned from `d[0]`, although its mathematical global index is k. Unlike de Casteljau interpolation, the weights depend on both the parameter and the knots.

2.4 B-spline derivatives and surfaces

The function `EvaluateBSplineDerivation` receives the original control points, knots, degree, count, and parameter. For positive degree, I construct

$$\mathbf{E}_i = \frac{p}{U_{i+p+1} - U_{i+1}}(\mathbf{P}_{i+1} - \mathbf{P}_i), \tag{8}$$

where $i = 0, \ldots, N - 2$. The derivative curve has degree $p - 1$ and $N - 1$ control points. I remove the first and last knots and evaluate the new curve with `EvaluateBSplineCurve`. If a derivative-control-point denominator is zero, my helper stores a zero vector instead of dividing by zero.

In `EvaluateBSpline`, I pass the surface and the parameters u, v. I use the same tensor-product organization as in the Bézier implementation. I first evaluate fixed-i rows using the v knots and degree, then evaluate the intermediate points using the u knots and degree. This gives `position`; evaluating the corresponding derivative curve gives `du`. Reversing the directions provides the intermediate points for `dv`.

I obtain the normal from `du.Cross(dv)`. For degenerate samples, `poleVectorBSpline` searches nearby valid parameters. Its steps are scaled independently:

$$\begin{aligned} h_u &= 10^{-4}(\texttt{EndU} - \texttt{BeginU}), \\ h_v &= 10^{-4}(\texttt{EndV} - \texttt{BeginV}). \end{aligned} \tag{9}$$

I check both domain coordinates before each candidate evaluation. This matters when u and v have different domain lengths.

A Bézier patch is a special B-spline surface with the endpoint knots repeated $p+1$ and $q+1$ times and no interior knots. Therefore, the same Bézier data can also be evaluated by my B-spline functions. The `B` key selects this CPU path without changing the degrees or control points.

2.5 Sampling and constructing triangles

The function `TessellateBezier` takes `patch`, `resolution`, and `method`, and samples the unit parameter domain. I implemented the evaluation path. The supplied `method` parameter does not dispatch to subdivision in my implementation.

Let r = `resolution`. I sample $(u_i, v_j) = (i/r, j/r)$ for $i, j = 0, \ldots, r$, producing $(r + 1)^2$ vertices. Each `Vertex` stores the evaluated position and unit normal.

I flatten the grid with $k(i, j) = i(r+1)+j$. For each cell, its corners are

$$\begin{aligned} a &= k(i, j), & b &= k(i + 1, j), \\ c &= k(i + 1, j + 1), & d &= k(i, j + 1). \end{aligned} \tag{10}$$

I add triangles (a, b, c) and (a, c, d). Their order follows increasing u and then increasing v, matching the normal orientation. Each patch contains $2r^2$ triangles.

`TessellateBSpline` takes a surface and a resolution. I construct the two sample arrays independently. Their names are `uSamples` and `vSamples`. Along u, I visit indices from `degreeU` through `countU-1`. Each positive-length span contributes

$$U_k + \frac{j}{r}(U_{k+1} - U_k), \qquad j = 0, \ldots, r - 1. \tag{11}$$

I skip zero-length intervals and append `EndU()` once at the end. I do the same along v. Thus, the resolution subdivides each nonempty span; it is not the total subdivision count for the entire B-spline domain.

For sample counts N_u and N_v, the resulting grid contains $N_u N_v$ vertices and $2(N_u - 1)(N_v - 1)$ triangles. Its flattened stride is N_v, which may differ from the Bézier stride.

2.6 Adaptive resolution from camera distance

I added `RelativeDistance` and `SelectResolution` in `Main.cpp`. The supplied `Bounds` function gives the control point bounding-box center $\mathbf{c}$ and half-diagonal length R. I cache these values when reframing the camera. For camera position $\mathbf{e}$, I use

$$d = \frac{\|\mathbf{e} - \mathbf{c}\|}{R}. \tag{12}$$

Normalizing by model size makes the distance rule comparable between models.

The selected resolution is

$$r(d) = \begin{cases} 16, & d < 3, \\ 8, & 3 \le d < 5, \\ 4, & 5 \le d < 7, \\ 1, & d \ge 7. \end{cases} \tag{13}$$

Closer views receive a denser grid. This is a model-wide distance rule. I do not use curvature estimation or irregular triangulation.

I save the relative distance of the most recent actual resolution change. If the current distance differs from it by less than 0.4, `SelectResolution` returns zero, meaning that the current resolution is retained. I update this saved distance only when a nonzero result actually changes `state.resolution`. I then set `remesh` to request reconstruction. This is a dead zone around the last change distance. The cached bounds are recalculated on reframing, rather than after every control point edit.

2.7 Selecting and dragging control points

I implemented screen-space picking with `ProjectToNDC`. It receives a world point, view matrix, and projection matrix. It writes an NDC position to an output parameter and returns whether the point passes the clip-region checks.

For a world point $\mathbf{p}$, I calculate $\mathbf{c} = PV(\mathbf{p}, 1)^{\mathsf{T}}$. I reject nonpositive c_w and coordinates outside the clip bounds. For an accepted point,

$$\mathbf{n} = (c_x, c_y, c_z)/c_w. \tag{14}$$

With window dimensions W, H and a top-left cursor origin, I obtain

$$s_x = (n_x + 1)W/2, \qquad s_y = (1 - n_y)H/2. \tag{15}$$

I use `glfwGetWindowSize` to match the cursor coordinate system. Framebuffer dimensions are used for rendering.

I select only on a fresh right-button press. I visit every surface and its `countU * countV` control points. For cursor position (x, y), I compare the squared distance

$$D = (s_x - x)^2 + (s_y - y)^2. \tag{16}$$

The current `err` is 6 and is compared directly with D, so the effective radius is $\sqrt{6}$ window units. I choose the accepted candidate with the smallest screen distance. This method does not perform depth-buffer occlusion tests, so overlapping point projections can be ambiguous.

The selected `NPCPoints` record stores the surface index, control point indices, and NDC position. While the right button is held, I apply

$$\begin{aligned} n_x &\leftarrow n_x + 2\Delta x/W, \\ n_y &\leftarrow n_y + 2\Delta y/H, \end{aligned} \tag{17}$$

where $\Delta x = x_{\text{current}} - x_{\text{previous}}$ and $\Delta y = y_{\text{previous}} - y_{\text{current}}$. I keep n_z fixed. With unchanged camera and projection matrices, the point moves in a plane parallel to the image plane.

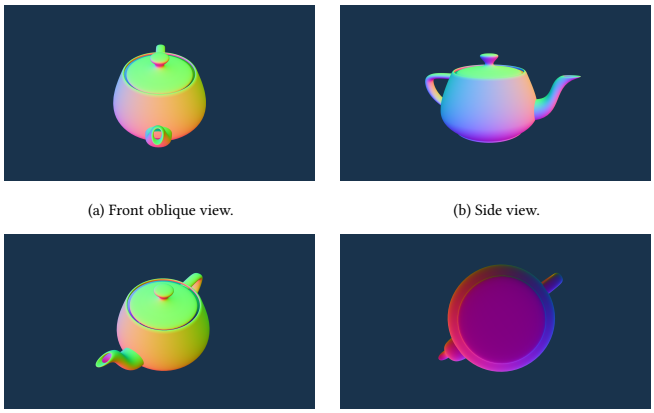
The function `NDCToWorld(ndc,view,projection)` calculates

$$\begin{aligned} \mathbf{h} &= (PV)^{-1}(n_x, n_y, n_z, 1)^{\mathsf{T}}, \\ \mathbf{p}_{\text{new}} &= (h_x, h_y, h_z)/h_w. \end{aligned} \tag{18}$$

The final division is necessary because $\mathbf{h}$ is homogeneous. I do not need to recover the original clip-space w separately.

I obtain a mutable reference from `Surface::Point(i,j)` and assign the new position to it. A nonzero cursor displacement sets `remesh`, so reconstruction uses the edited data. Releasing the right button clears selection. Orbiting, scrolling, switching models, or resetting the camera also cancels it.

This editor changes one stored control point. It does not synchronize coincident points across patches or enforce periodic constraints. Therefore, editing a shared boundary can change continuity.



(a) Front oblique view. (b) Side view.

(c) Upper view. (d) Bottom surface of model 1.

Fig. 1. Planned views of model 1, the Bézier teapot with a bottom.

2.8 Evaluating Bézier patches on the GPU

I implemented the calculations in `Bezier.tese` and use the supplied `PatchMesh` path. The CPU uploads patch control points rather than all sampled surface vertices. `Patch.vert` passes them to the tessellation evaluation shader.

The supplied draw function sets the patch size using the constant `GL_PATCH_VERTICES`. It then draws with `GL_PATCHES`. The provided bicubic patches have $4 \cdot 4 = 16$ points each. This path uses the default inner and outer tessellation levels supplied by the application, so it has no tessellation control shader.

I read u, v from `gl_TessCoord.xy` and evaluate the position and partial derivatives with the same de Casteljau construction as on the CPU. I output the world position and unit normal, and set

$$\texttt{gl_Position} = PV(\mathbf{S}(u, v), 1)^{\mathsf{T}}. \tag{19}$$

The fragment stage remains `Surface.frag`.

My one-dimensional working arrays use `MAX_CURVE_POINTS = 4`, sufficient for the provided cubic curves. The complete patch still contains 16 points, so its input capacity is separate from these curve arrays. This shader does not implement arbitrary degrees or GPU B-spline evaluation.

At a degenerate point, the shader searches nearby samples. If every candidate fails, it returns $(0, 1, 0)$ as a display fallback. This differs from the CPU exception and is not a recovered geometric normal. The `G` key switches Bézier models between CPU and GPU paths. Both evaluate the same mathematical patch, while the hardware tessellator determines its own triangulation.

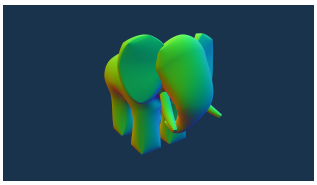
3 RESULTS

3.1 The teapot with a bottom: model 1

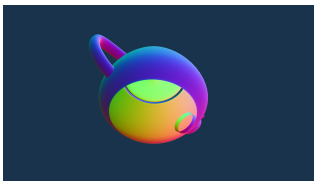
Figure 1 reserves four views of model 1, the complete Bézier teapot with a bottom, in normal-as-color mode.

3.2 The elephant, the second teapot, and the face

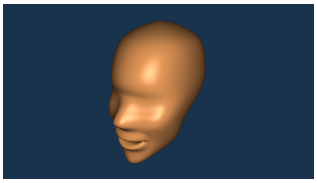
Figure 2 reserves images of model 2, the Bézier elephant; model 3, the B-spline teapot without a bottom; and model 4, the B-spline face.



(a) Model 2: Bézier elephant.



(b) Model 3: teapot without a bottom.



(c) Model 4: face, shaded.



(d) Model 4: face, normal colors.

Fig. 2. Planned renders of models 2–4 and face normal colors.



(a) CPU Bézier evaluation.



(b) CPU B-spline evaluation.

Fig. 3. Planned CPU evaluator comparison on model 1.

3.3 Two CPU evaluators on the same patch data

Both panels of Figure 3 use model 1, the teapot with a bottom. In CPU mode, I will keep the camera, control net, and displayed resolution fixed and toggle B. This compares direct Bézier evaluation with its special-case B-spline interpretation. Both panels retain the same model and degrees.

3.4 Adaptive mesh density

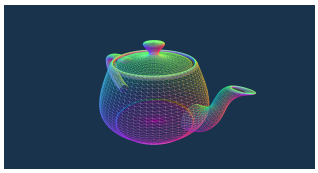
Figure 4 reserves wireframe captures of model 1 at all four resolution levels. I will change camera distance and record the actual resolution in the title. Since zoom changes the model's screen size, the comparison should consider both triangle density and the reported level.

3.5 Interactive editing

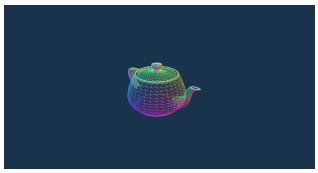
For Figure 5, I will use model 2, the elephant, enable its control net, and capture the original state, a moderate displacement, and a larger deformation after release. The shading will remain unchanged. An isolated control point avoids ambiguity from overlapping projections.

3.6 CPU and GPU rendering

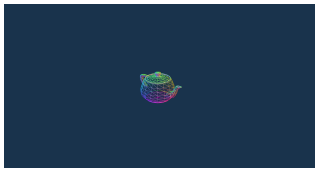
Both panels of Figure 6 use model 2, the elephant. I will capture it before and after toggling G, with the normal-color mode, and displayed resolution fixed. The images will compare the rendered surface; they will not establish identical triangle layouts or a measured speedup.



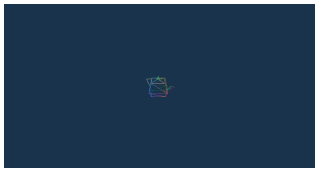
(a) Near, resolution 16.



(b) Resolution 8.

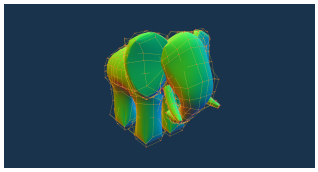


(c) Resolution 4.

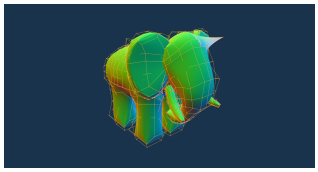


(d) Far, resolution 1.

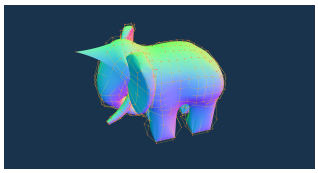
Fig. 4. Planned distance-based wireframe examples.



(a) Original control net.

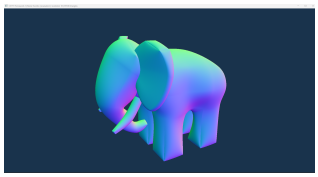


(b) Moderate displacement.

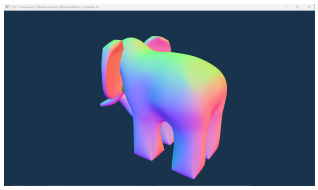


(c) Larger displacement, after release.

Fig. 5. Planned control point editing sequence on the elephant.



(a) CPU surface sampling.



(b) GPU tessellation evaluation.

Fig. 6. Planned CPU/GPU comparison on the Bézier elephant.

During development, the TES produced temporary-array resource allocation errors. I reduced the one-dimensional working-array capacity to four to match the supplied cubic curves.